Lecture8: Clustering

STA 4724

HanQin Cai

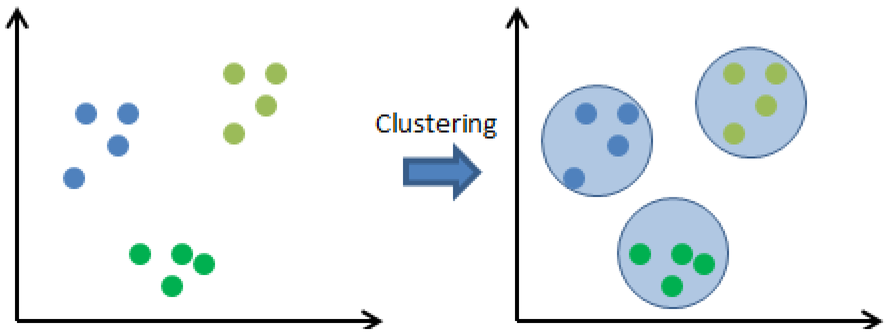
University of Central Florida

Clustering

- The process of dividing a given dataset into several groups (i.e., clusters)

- Data points in the same cluster are as similar as possible

- Data points in different clusters are dissimilar



- Clustering requires data, but not predefined labels
  - Unsupervised learning

- When the predefined label does exist, we should discuss *classification* instead (cover in later classes)

Clustering is Natural to Human

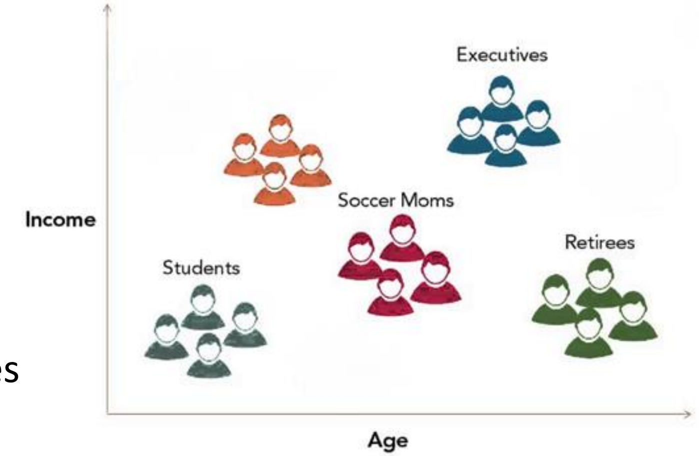
One corner in the store

- Imagine you are managing a supermarket
- Naturally, you put all the seafood in one section
- The meat section is usually not too far away
- TVs and video games are in another corner of the store

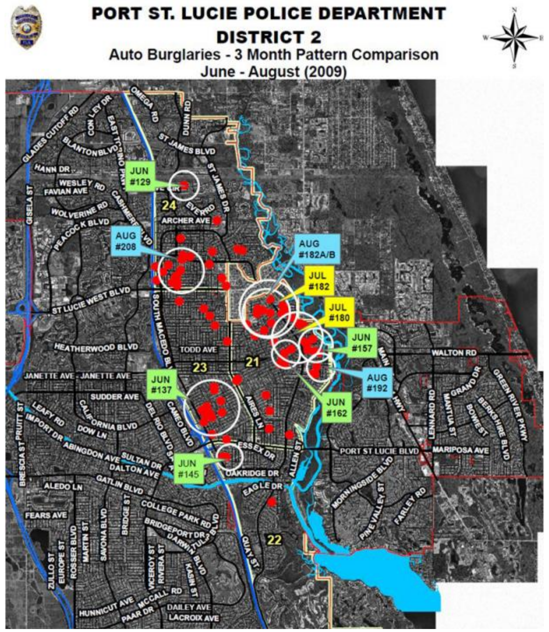
Another corner

Applications

- Many applications in computer science, market research, social science, biology, medical imaging …



- Two examples:
  - Customer segmentation
    - ❖ Group customs based on their backgrounds and shopping styles
    - ❖ The business owner can apply specialized marketing strategies



  - Crime analysis
    - ❖ Identify the "hot spots" where a similar crime has happened over a period of time
    - ❖ Help the law enforcement officials to manage their resources more effectively

Toward Supervised Learning

- Clustering is therefore a data exploration technique that contributes to our familiarity with the dataset

- Once suitable clusters have been created, we can provide them labels based on human knowledge

- These clusters can be the starting point to use supervised machine learning techniques

K-Means Clustering

- Clustering is NP-hard (non-deterministic polynomial-time hard)
  - Discussion on board
  - Finding the best solution is very computationally difficult

- K-Means is an efficient heuristic algorithm for clustering
  - Converges quickly to a local solution

- "K" in K-Means: the number of clusters

K-Means Clustering

- Iteratively assigns each point to one of the *K* clusters
  - Based on how close the point to the current means of the clusters

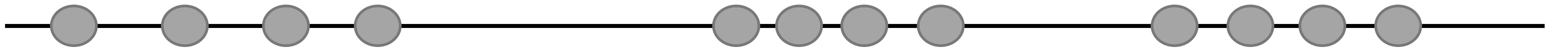
- A greedy algorithm
  - It is a heuristic algorithm that always takes the local optimal choices
  - the final solution depends on the initial conditions

- In general, a heuristic helps us solve a problem in a quick fashion by finding an approximate solution

Toy Example

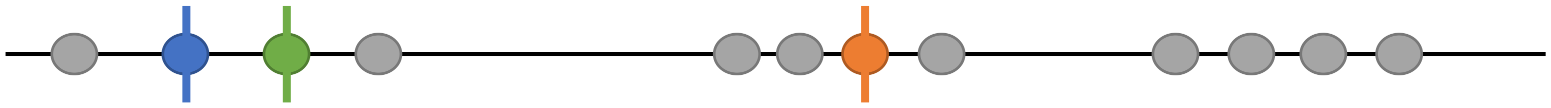
- Image we have some data points that we can plot on a line

- Visually, we know we want to put them into 3 clusters, i.e., K = 3



Toy Example: Step by Step

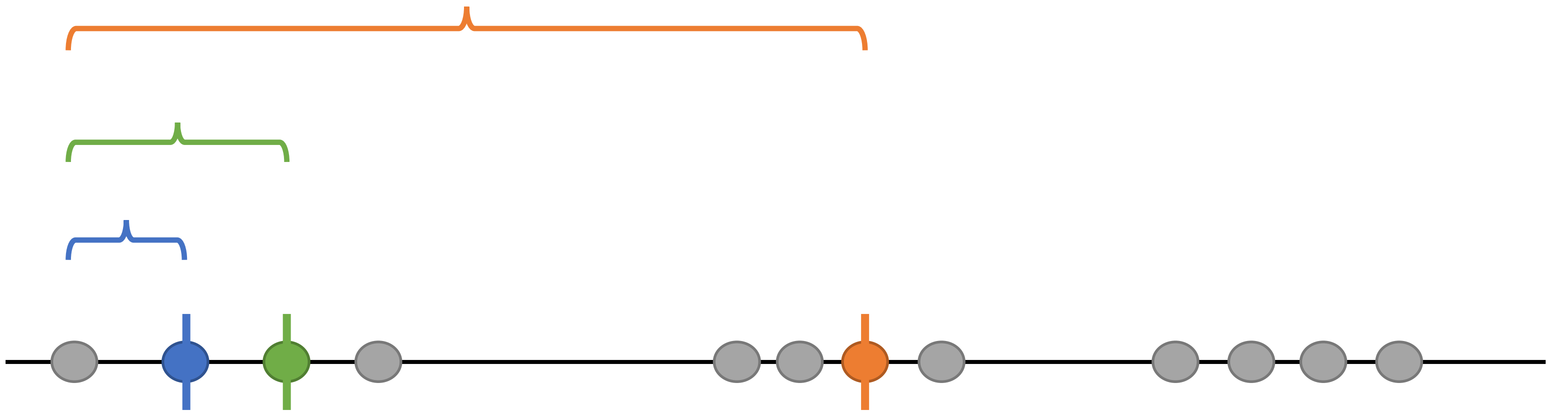
1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now



| denotes the mean of the **blue** cluster

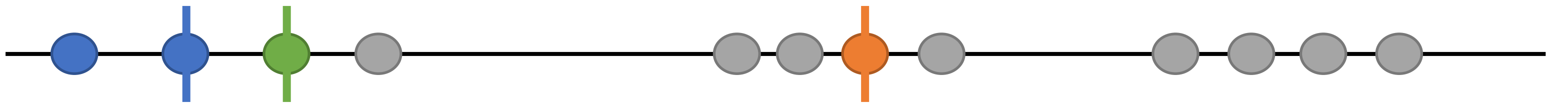
Toy Example: Step by Step

1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now
2. Measure the distance between the 1st point and the means of the 3 clusters



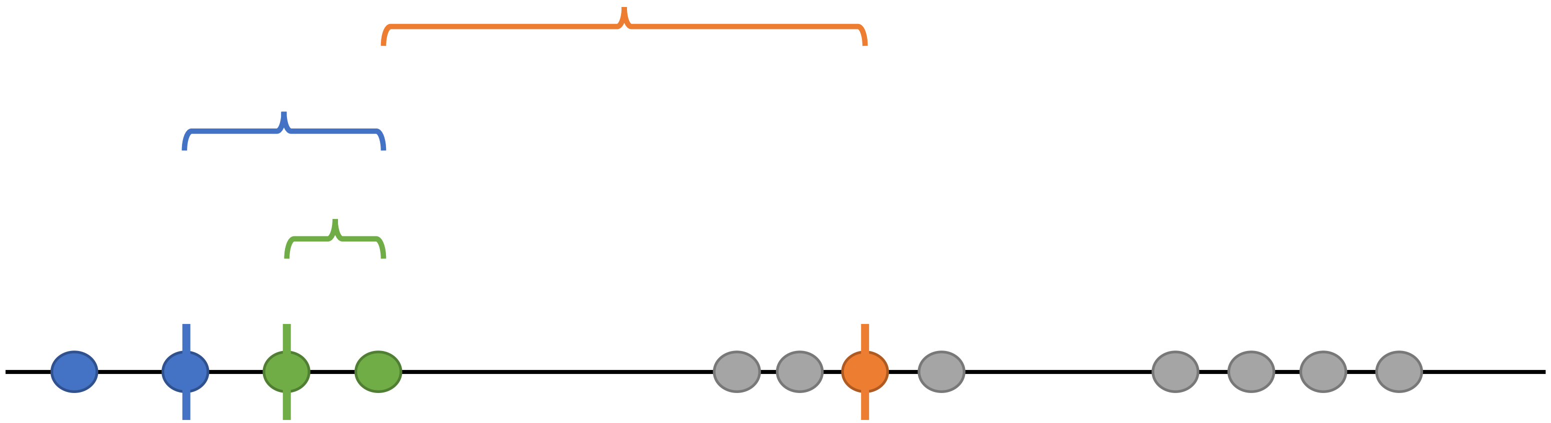
Toy Example: Step by Step

1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now
2. Measure the distance between the 1st point and the means of the 3 clusters
   - Assign the 1st point to the nearest cluster (**blue**, in this case)



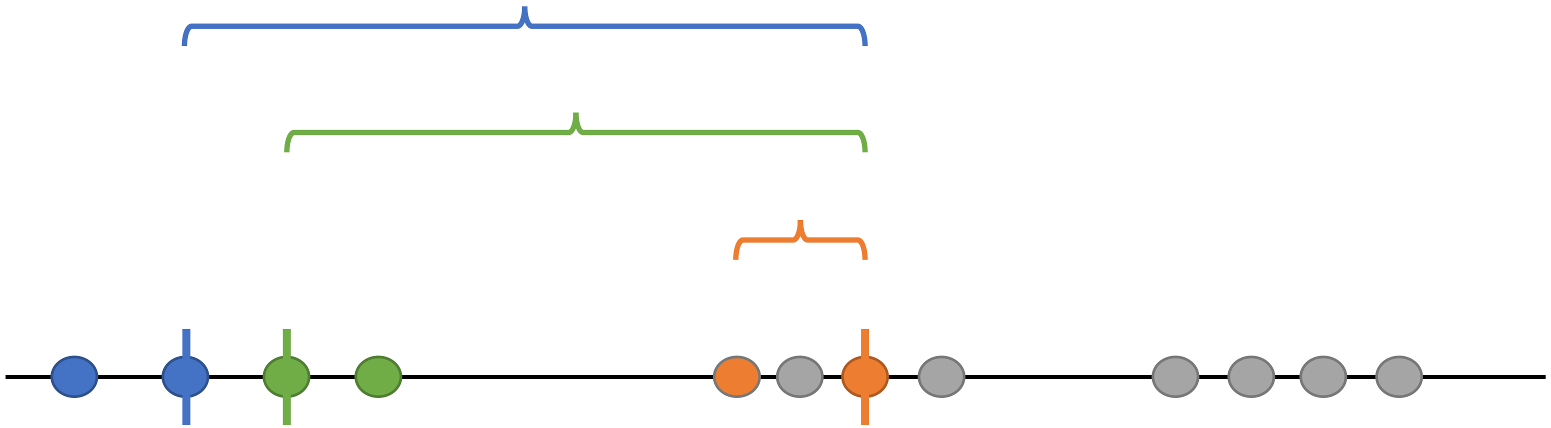
Toy Example: Step by Step

1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now
2. Measure the distance between the 1st point and the means of the 3 clusters
   - Assign the 1st point to the nearest cluster (**blue**, in this case)
3. Use the same method to assign every point to one of the clusters



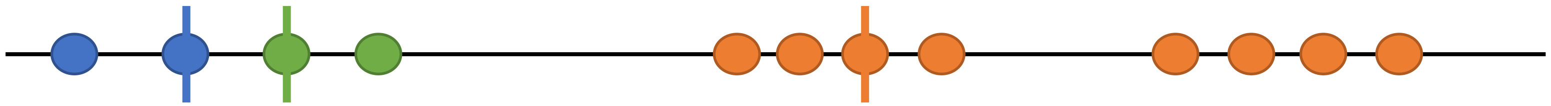
Toy Example: Step by Step

1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now
2. Measure the distance between the 1st point and the means of the 3 clusters
   - Assign the 1st point to the nearest cluster (**blue**, in this case)
3. Use the same method to assign every point to one of the clusters



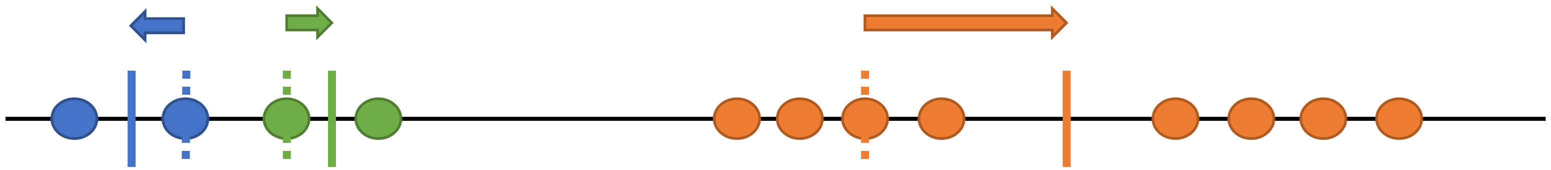
Toy Example: Step by Step

1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now
2. Measure the distance between the 1st point and the means of the 3 clusters
   - Assign the 1st point to the nearest cluster (**blue**, in this case)
3. Use the same method to assign every point to one of the clusters



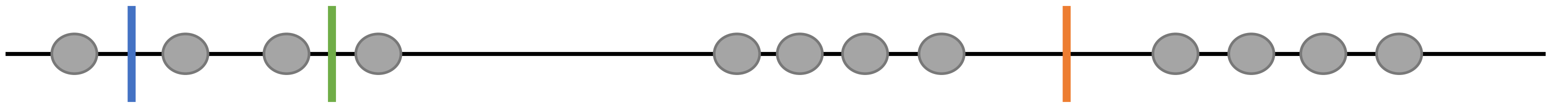
Toy Example: Step by Step

1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now
2. Measure the distance between the 1st point and the means of the 3 clusters
   - Assign the 1st point to the nearest cluster (**blue**, in this case)
3. Use the same method to assign every point to one of the clusters
4. Update the means of the clusters



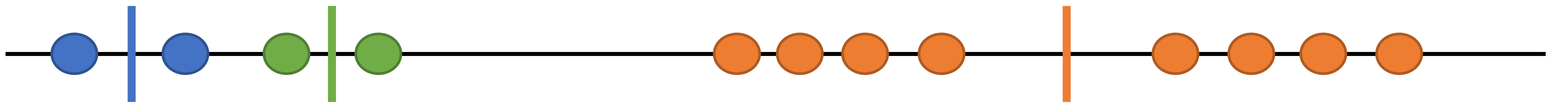
Toy Example: Step by Step

1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now
2. Measure the distance between the 1st point and the means of the 3 clusters
   - Assign the 1st point to the nearest cluster (**blue**, in this case)
3. Use the same method to assign every point to one of the clusters
4. Update the means of the clusters
5. Reassign all the points again, using the **new** means



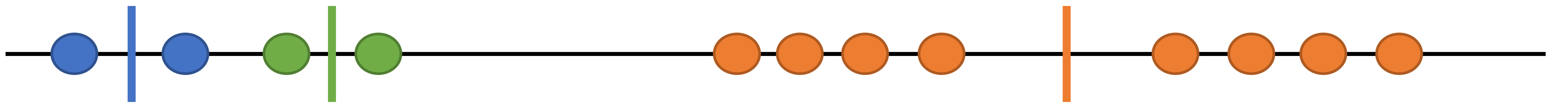
Toy Example: Step by Step

1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now
2. Measure the distance between the 1st point and the means of the 3 clusters
   - Assign the 1st point to the nearest cluster (**blue**, in this case)
3. Use the same method to assign every point to one of the clusters
4. Update the means of the clusters
5. Reassign all the points again, using the **new** means



Toy Example: Step by Step

1. Initial 3 clusters by randomly selecting 3 distinct data points
   - They become the means of the clusters, for now
2. Measure the distance between the 1st point and the means of the 3 clusters
   - Assign the 1st point to the nearest cluster (**blue**, in this case)
3. Use the same method to assign every point to one of the clusters
4. Update the means of the clusters
5. Reassign all the points again, using the **new** means
6. Repeat Steps 4-5 until the algorithm converges
   - convergence: no changes in the points assignment anymore



- So, we get a solution for this clustering problem

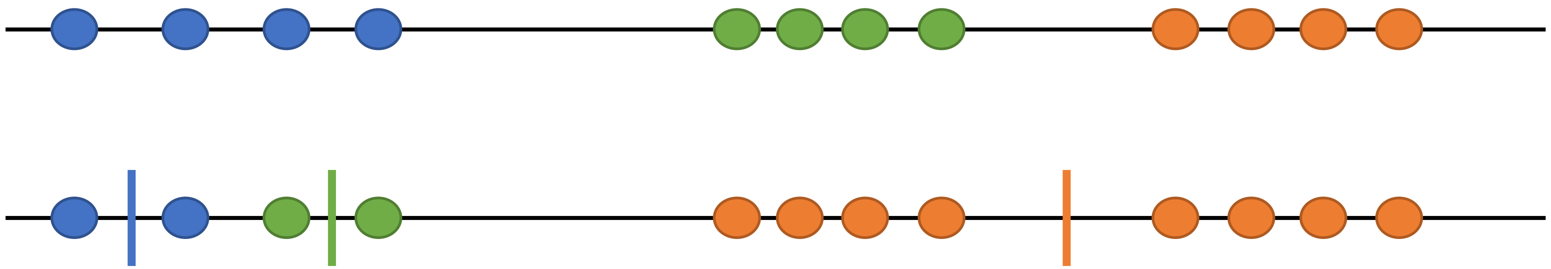
But…

- This solution is not matching the human solution

- Not a surprise --- Recall that *K*-Means does **not** guarantee convergence to the best solution

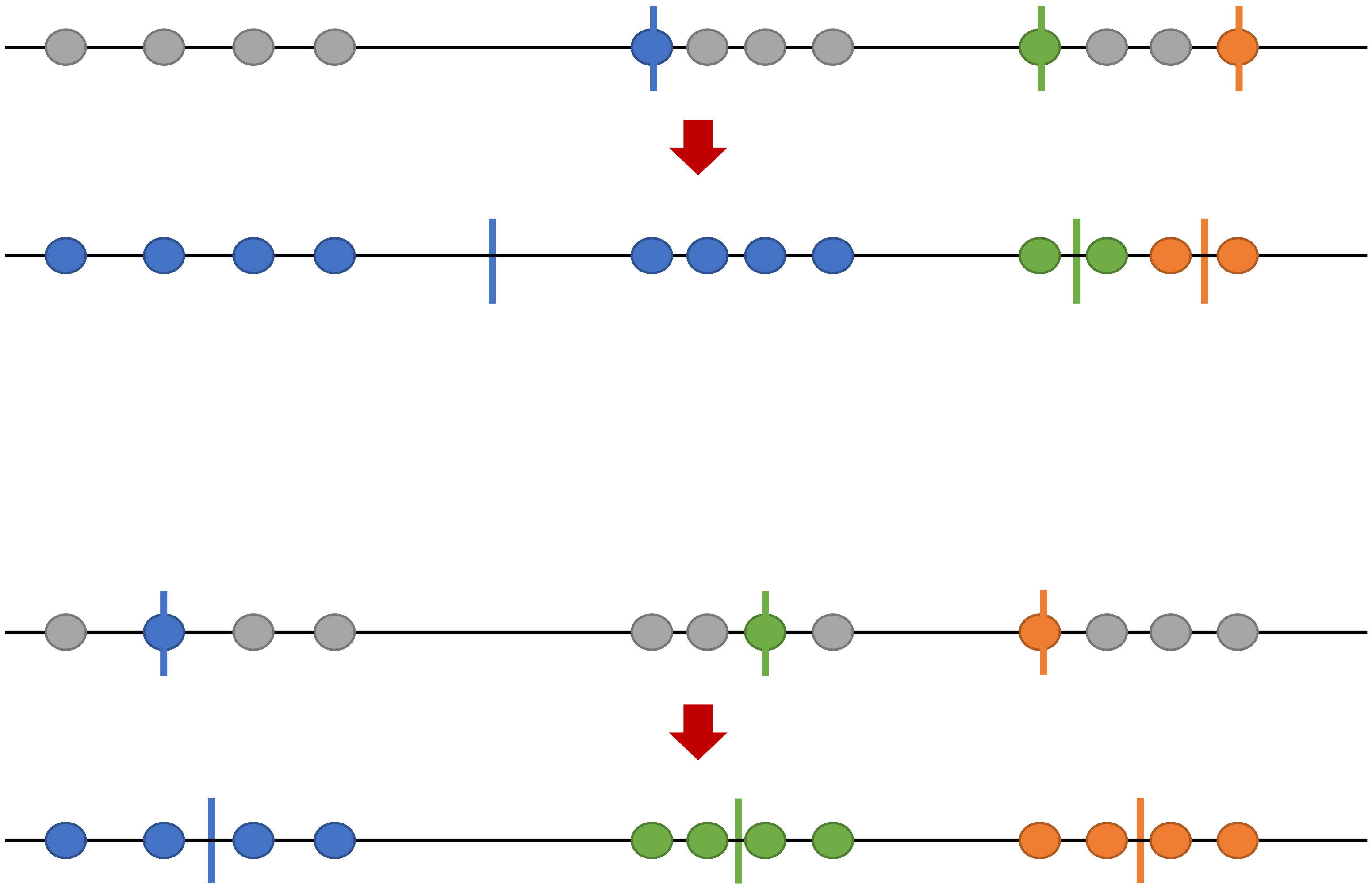
- Try *K*-Means with several different initializations, then pick the best result

Human solution →



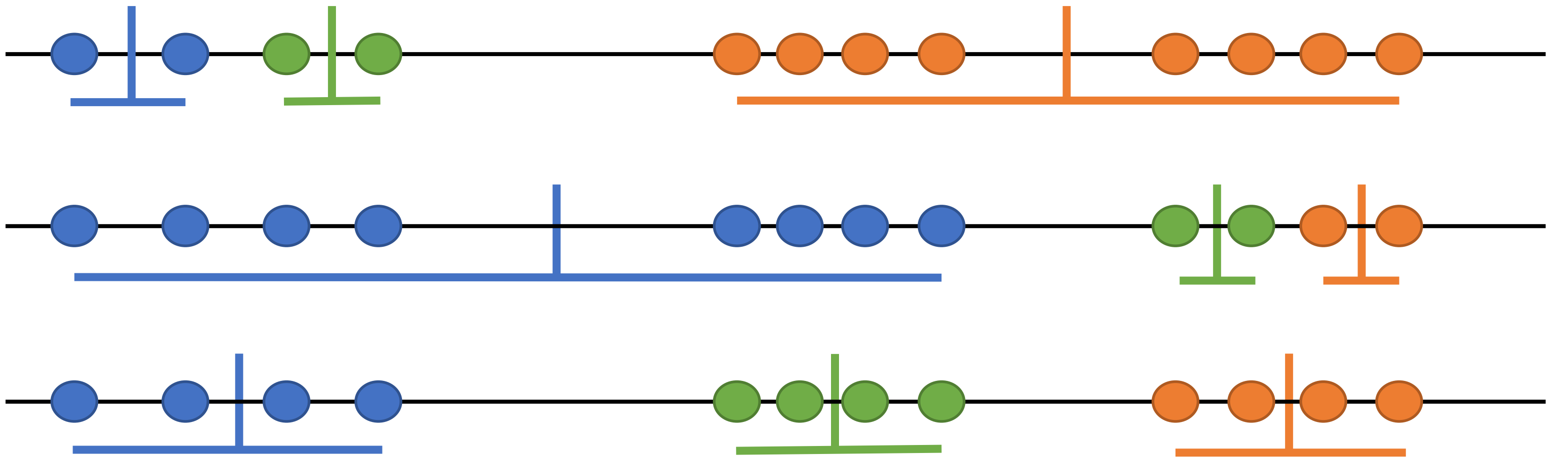
More solutions

- Different initializations may give different clustering solutions



Clustering Quality

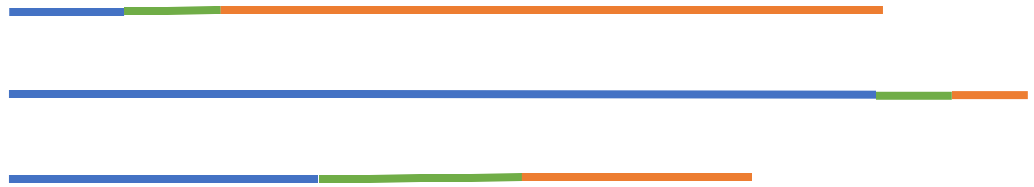
- Question: How to measure the quality of a solution?
- Minimize the sum of variation within each cluster (aka. *total intra-cluster variation*)



1st solution:

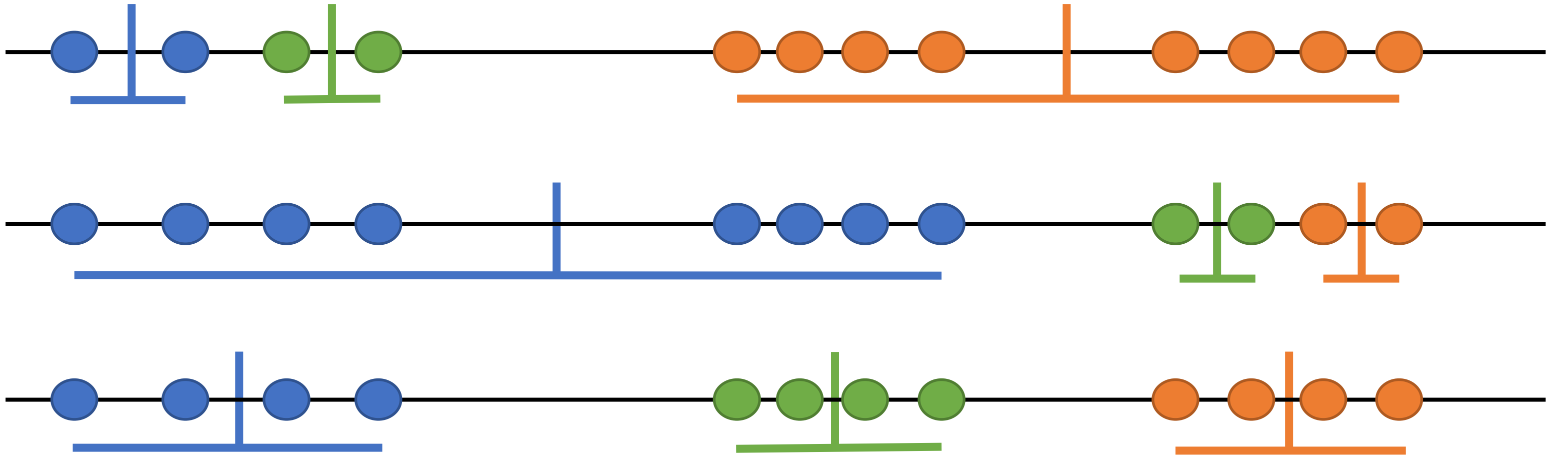
2nd solution:

3rd solution:



Clustering Quality

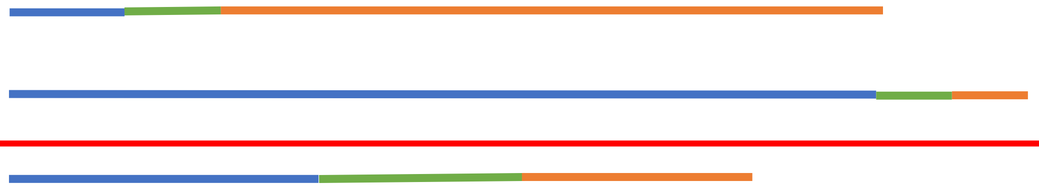
- Question: How to measure the quality of a solution?
- Minimize the sum of variation within each cluster (aka. *total intra-cluster variation*)



1st solution:

2nd solution:

3rd solution:



← The best solution so far

- Computer still doesn't know if this is the best solution overall, try more rounds!

One More Challenge

- In this example, by visualizing the data, it is obvious that we have 3 clusters

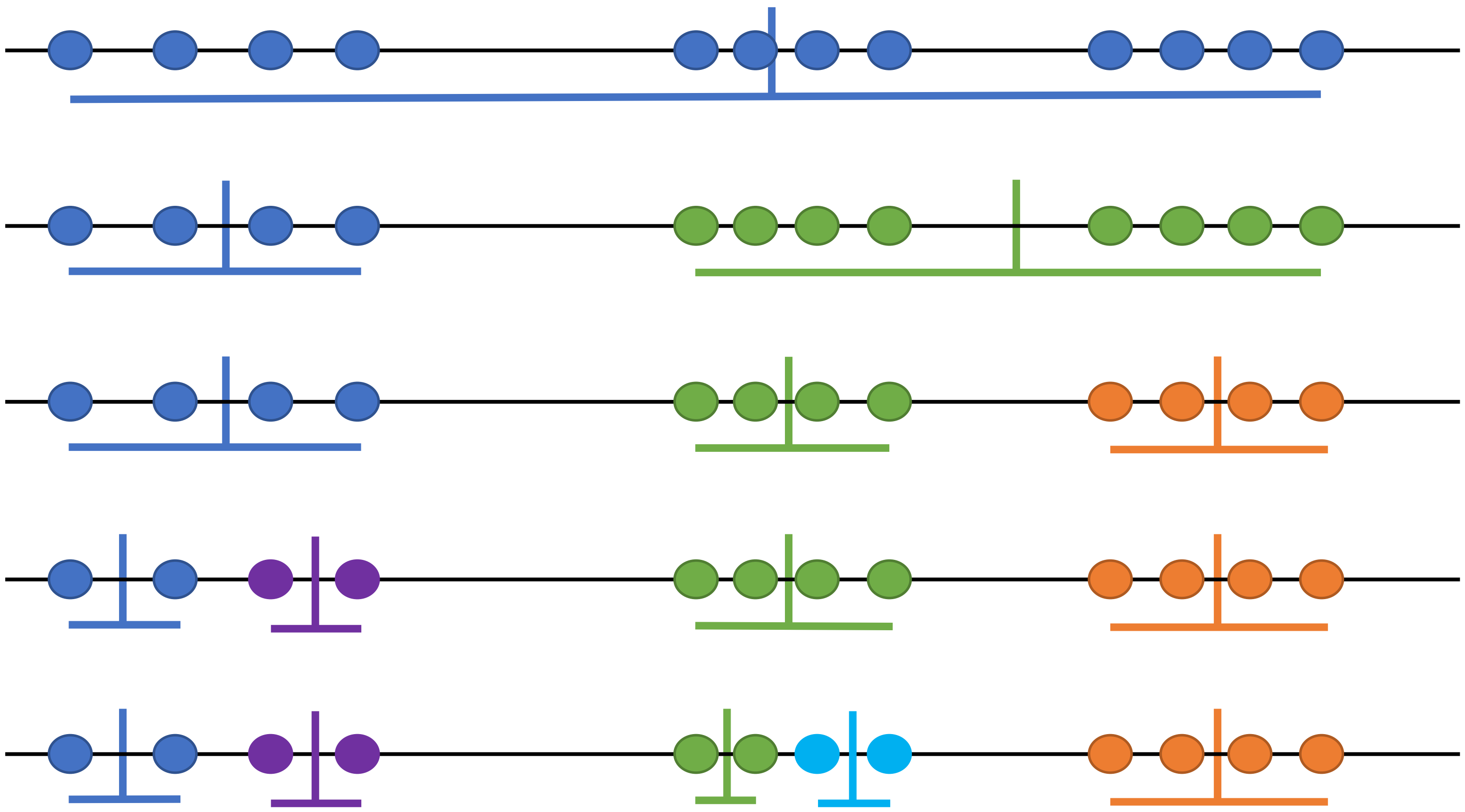
- Sometimes, it is not so clear how many clusters should be used
  - Especially, for high-dimensional dataset

- Question: How to mathematically find the best value for "*K*" (i.e., # of clusters)?

- Try different values for *K*, and keep track of the total intra-cluster variation

Find the Best Value for K

- $K = 1$
- $K = 2$
- $K = 3$
- $K = 4$
- $K = 5$



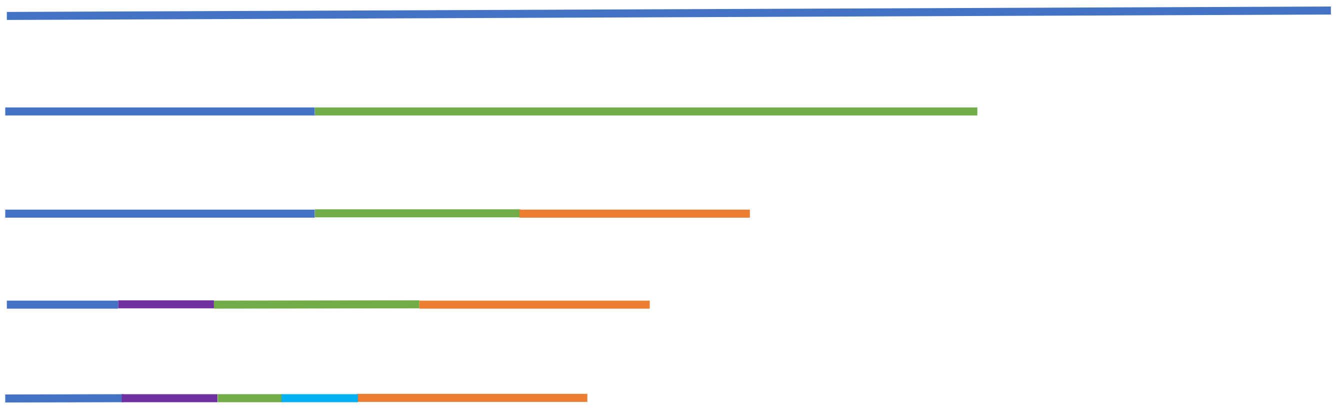
⋮

⋮

- $K = 12$ Each point is its own cluster

Find the Best Value for K

- $K = 1$
- $K = 2$
- $K = 3$
- $K = 4$
- $K = 5$



⋮

- Each time we add a new cluster, the total intra-cluster variation is getting smaller

⋮

- $K = 12$ Each point is its own cluster => Variation = 0

- However, $K = 12$ (# of the points) is obviously a **bad** choice for clustering
  - We are not getting any useful information

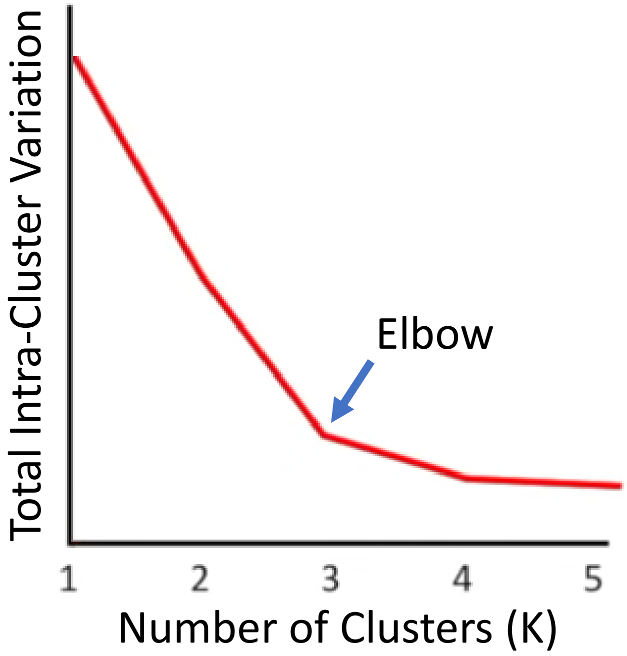
Elbow Method

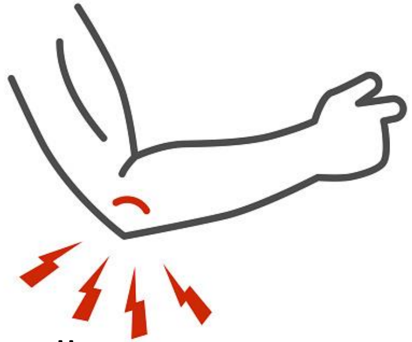
- Make a plot for the **total intra-cluster variation**

- Before K = 3, we see big reductions in variation. After that, the variation does not go down as quickly
  - K = 3 is called an "elbow" in this example

- Pick the best value for "K" by finding the "elbow" in the plot

- Finally, we have all the pieces for a complete K-Means algorithm





Elbow

More Complicated Examples

- Visualizing K-Means Clustering

High-Dimensional Data

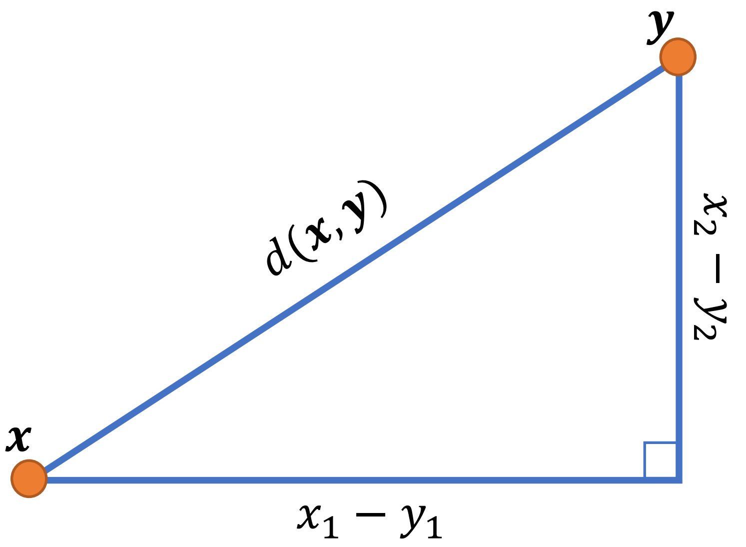
- While our toy example was on 1-D data, *K*-Means works for high-dimensional data

- The similarity between two high-dimensional points is measured by **Euclidean distance**

- Recall the definition from linear algebra class:
  - Given two n-D points: $\boldsymbol{x} = [x_1, x_2, \cdots, x_n]$ and $\boldsymbol{y} = [y_1, y_2, \cdots, y_n]$
  - Euclidean distance:

$$d(\boldsymbol{x}, \boldsymbol{y}) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + \cdots + (x_n - y_n)^2}$$

  - Physically, this is the **length** of the **straight line** between $\boldsymbol{x}$ and $\boldsymbol{y}$



- Now, we can use *K*-Means for high-dimensional datasets

Time to Play

- Let’s see how to use *K*-Means in **python**!